

FORGE — FDE Academy · D5 · 29-Jun-2026

# Azure for Enterprise AI

Foundry, Azure OpenAI, and the secure foundation — landing zones and identity — that lets a regulated enterprise run AI without leaking data or losing control.

RBAC

Azure AI Foundry

Azure OpenAI

PTU vs PAYG

landing zones

Managed identity

Azure vs OpenAI direct

Azure AI landscape

# HOW TO READ THIS HOUR

## One lens for every Azure decision — and one thread through all of them

We will examine every choice today through the same four questions you used on the scaffolding day. **What** it is · **Why** it exists · **How** to set it up · **When** to reach for it. Underneath them all runs a single thread: **identity**.



### WHAT

A plain definition — the service, the setting, the pattern — with every Azure term explained on first use.



### WHY

The enterprise problem it solves: compliance, data residency, cost control, reliability.



### HOW

The concrete setup — the portal step, the role, the CLI line, the deployment type.



### WHEN

The decision rule — and just as important, when NOT to use it.



### The thread: identity-bound everything

Foundry components authenticate with managed identities · Azure OpenAI should be keyless · the landing zone is identity-bound by design · IAM is the whole game. Hold this thread and the six topics become one story.

# THE ROUTE

## Six stops — landscape to a real decision



### 1 • Azure AI landscape

Services, pricing, SLAs, regional availability — the lay of the land.



### 2 • Azure AI Foundry

Projects, model catalogue, prompt flow, evaluation — the build platform.



### 3 • Azure OpenAI Service

Model selection, PTU vs PAYG, endpoints, quota — the engine room.



### 4 • Landing zones

Identity-bound design, network egress, data residency — the foundation.



### 5 • Managed identities & RBAC

Never use API keys — identity is the perimeter.



### 6 • Azure OpenAI vs direct OpenAI

Compliance, residency, cost, latency — the real-world call.

*Stops 1–3 are the toolkit • 4–5 are the foundation • 6 is where it all pays off in a single decision.*



TECHADEMY

## STOP 1

# The Azure AI Landscape

Services · pricing · SLAs · regional availability

*Know the terrain before you build on it.*



# STOP 1 · WHAT'S OUT THERE

## The map — and the names that keep changing

Azure's AI stack has three layers. Know which layer you're standing on.



### PLATFORM

**Microsoft Foundry**

The unified build platform: model catalogue, prompt flow, evaluation, agents, governance. Formerly Azure AI Studio → Azure AI Foundry.



### MODELS

**Azure OpenAI + 1,900 more**

GPT-5 / 4.1 / 4o / o-series via Azure OpenAI, plus Llama, Phi, Mistral, Cohere, Claude — all under one catalogue.



### SUPPORTING SERVICES

**Search · Safety · Document**

AI Search (RAG), Content Safety (guardrails), Document Intelligence, Speech, Vision — the parts that surround the model.

**Naming reality:** Azure AI Studio → Azure AI Foundry → now branded **Microsoft Foundry**. Azure OpenAI is now **Azure OpenAI in Foundry Models** — and an existing Azure OpenAI resource can be upgraded in place, keeping its endpoint.

## What you pay for — and what the SLA does NOT promise



### Pricing — the shape, not the cents



Per-token rates are roughly the SAME as OpenAI direct — same models, same labs.



Two modes: pay-as-you-go (per token) or PTU (reserved capacity). Stop 3 goes deep.



Cheaper levers: smaller models (mini / nano), prompt caching, Batch API (~50% off, 24h).



Hidden add-ons push total cost 15–40% higher: Private Link, Log Analytics, support plans.



No permanent free tier. Always confirm live rates on the Azure pricing page.



### SLA — read the small print

**99.9%** financially-backed monthly AVAILABILITY SLA



Covers: the inference endpoint being up — PayGo Standard and Provisioned Managed.



Does NOT cover latency or output quality. A 429 on every call still counts as 'available'.



Latency SLA exists only on Provisioned-Managed deployments — buy PTU for speed guarantees.



Credits are tiered (10/25/100%) and YOU must claim within 30 days with your own telemetry.

## Deployment scope = the dial between capacity and data residency

Same per-token price across scopes. You are trading capacity and latency-spread for how tightly data is pinned — not money. Pick the narrowest scope your compliance team needs.



### Global

Widest

Routed across the worldwide pool. Highest throughput, broadest model choice — NO residency guarantee.



### Data Zone

Balanced

Stays within a geography — US or EU. Cross-region load-balancing inside that boundary.



### Regional

Strictest

Processing and storage pinned to one region. Strictest residency, smallest capacity pool.

**Residency arrow: Global → Data Zone → Regional (capacity shrinks, control tightens)**

*This single dial — plus your choice of Azure region — is how Azure answers GDPR, residency and sovereignty questions. It returns at Stop 4 (landing zone) and Stop 6 (the decision).*



TECHADEMY

STOP 2

# Azure AI Foundry

Projects · model catalogue · prompt flow · evaluation

*The platform where you actually build, test and govern.*





## Foundry is the vehicle; Azure OpenAI is one engine inside it

**Azure AI Foundry (Microsoft Foundry)** is the unified platform for building, evaluating, governing and deploying AI apps and agents. Azure OpenAI is **one component** of it. Foundry adds a 1,900-model catalogue, orchestration, evaluation, responsible-AI guardrails and one unified RBAC + networking namespace.



### Hub vs project — the unit of work

A hub holds shared compute, connections and security settings. A project is the workspace inside a hub where you actually build. New 'Foundry projects' are replacing classic 'hub-based projects' — when in doubt, create a Foundry project.



### The engine-and-vehicle test

If a task needs only chat completions, you may touch just the Azure OpenAI engine. If it needs RAG, multiple model families, evaluation or agents, you want the whole Foundry vehicle.

### One catalogue, many providers



#### Azure OpenAI

GPT-5 · 4.1 · 4o · o-series



#### Microsoft

Phi family (cost-efficient)



#### Open / partner

Llama · Mistral · Cohere



#### Frontier partners

Anthropic Claude · Hugging Face

*~1,900 models — one inference SDK, one RBAC model.*

# STOP 2 · THE BUILDING BLOCKS

## Four things Foundry gives you that raw API access doesn't



### Projects & hubs

Shared, governed workspaces — compute, connections, security and RBAC in one place, not scattered per developer.



### Model catalogue

Browse, compare and deploy ~1,900 models from one place; swap GPT-4o for Phi-4 to cut cost with a config change.



### Prompt flow

Visual + code DAG to chain LLM calls, Python and logic — Azure-native, with versioning and one-click deploy.



### Evaluation

Manual and automated scoring of output quality, groundedness and safety — testing for non-deterministic systems.



### Why this matters

The catalogue + prompt flow + evaluation are exactly the scaffolding ideas from D4 — typed contracts, orchestration and contract-testing — but managed for you, inside Azure's governance and identity boundary.

## Powerful — but mind the road sign ahead



### Prompt flow



A DAG where each node is a step: an LLM call, a Python function, an API call or a branch.



Conceptually like LangGraph, but Azure-native with built-in versioning, evaluation and deploy.



Use it for multi-step orchestration beyond a single API call.



### Evaluation



Manual evaluation: run a test set, eyeball and score responses against expected answers.



Automated evaluation: metrics for relevance, groundedness, coherence, safety — at scale.



This is your contract/eval gate (D4) made visual — wire it before you ship a flow.



### Road sign: Prompt flow is being retired

Microsoft has announced Prompt flow retirement on 20 April 2027 and no longer recommends it for NEW development — the successor is the Microsoft Agent Framework. Teaching value remains (the DAG mindset is universal), but for new production work, start on the Agent Framework, not classic Prompt flow. Always check the current Foundry docs before committing an architecture.

## When to reach for Foundry vs raw Azure OpenAI

@app.post("/complete") response: CompletionResponse



### Platform, not a model

Foundry wraps Azure OpenAI plus ~1,900 models in one governed, identity-bound place.



### Catalogue = right-sizing

Swap models by config — Phi for cheap tasks, GPT for hard ones, Claude when it fits.



### Orchestrate & evaluate

Prompt flow for chains, evaluation for quality — but build new work on Agent Framework.



### When to bother

Single chat call → just the engine. RAG, multi-model, agents, governance → the vehicle.

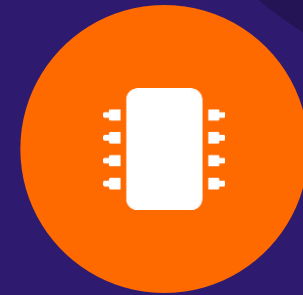
***One sentence: use Foundry when you need governance, multiple models, orchestration or evaluation — not just a single completion.***



TECHADEMY

STOP 3

# Azure OpenAI Service



Model selection · PTU vs PAYG · endpoints · quota

*The engine room — pick a model, an endpoint, and a way to pay.*

Choose the smallest model that works — then call it keyless



## WHAT

A model 'deployment' is a named instance of a model in your resource. You call the deployment name, not the raw model.



## WHY

Model choice is the single biggest cost and latency lever. A mini / nano model is often ~90% cheaper and good enough.



## HOW

Resource → deploy model → get an endpoint. Authenticate with Entra ID via DefaultAzureCredential — no API key.



## WHEN

Start small (Phi / mini) for classify, extract, route. Reach for GPT-5 / o-series only for hard reasoning.

Keyless call to a deployment — identity, not a secret

```
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
token = get_bearer_token_provider(DefaultAzureCredential(), "https://cognitiveservices.azure.com/.default")
client = AzureOpenAI(azure_endpoint=ENDPOINT, azure_ad_token_provider=token, api_version="2024-10-21")
client.chat.completions.create(model="gpt-4o-mini", messages=[...]) # model = deployment name
```

PTU vs Pay-As-You-Go — the same model, a very different bill

| Dimension   | Pay-As-You-Go (Standard)              | Provisioned Throughput (PTU)                            |
|-------------|---------------------------------------|---------------------------------------------------------|
| You pay for | Each token, as used                   | Reserved capacity per hour — used or not                |
| Latency     | Best-effort, varies under load        | Predictable; latency SLA on Provisioned-Managed         |
| Commitment  | None — turn it off any time           | Hourly, or monthly/annual reservation (cheaper)         |
| Best for    | Dev, spiky, unpredictable, low volume | Steady, high-volume production that can't tolerate 429s |
| Risk        | 429 throttling at peak; tail latency  | Paying for idle PTUs you over-provisioned               |



The decision rule (don't guess — measure)

Run PAYG for 30–60 days, measure your P95 hourly token throughput, then model PTU against the real numbers. PTU wins on steady, high-volume traffic at sustained utilisation above ~50%; below that, PAYG's flexibility wins. Annual reservations cut ~30% more and, in 2026, apply across the Foundry catalogue.

## Quota, capacity and the 429 — and how to survive a burst



### TPM & RPM (PAYG)

Tokens- and requests-per-minute caps, set per region, per subscription, per model. Exceed them → HTTP 429.



### PTU quota ≠ capacity

Quota is your max deployable PTUs (a policy limit, free). Capacity is whether the region can actually give them now.



### Survive the burst

Retry with back-off (your D4 habit). Spread quota across regions. Use spillover: overflow PTU traffic falls back to Standard.

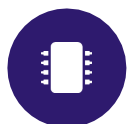


### The 429 is the same flaky-vendor problem from D4

A burst exhausts your TPM or PTU and the service returns 429 Too Many Requests — and remember, that still counts as 'available' under the SLA. The fix is the scaffolding you already built: typed client, retry-with-back-off, timeout. Then add Azure levers: request a quota increase, spread deployments across regions, and turn on spillover so PTU overflow lands on a Standard deployment instead of failing.



## The engine room in four lines



### Right-size the model

Smallest model that works is the biggest cost and latency win — re-check monthly.



### Call it keyless

Deployment name + Entra ID via DefaultAzureCredential. No secret in the call.



### PTU vs PAYG = measure

PAYG for spiky/dev; PTU for steady high volume. Decide on 30–60 days of real data.



### Plan for the 429

Quota ≠ capacity. Retry, back-off, multi-region and spillover keep you up under burst.

***One sentence: pick the smallest model, authenticate with identity, pay the way your traffic shape demands, and design for the 429.***



TECHADEMY

## STOP 4

# Landing Zones

Identity-bound design · network egress · data residency



*AI is just another workload — give it a foundation built before the first deploy.*

## A landing zone is the foundation poured before any workload arrives

An **Azure landing zone** is a pre-configured environment that sets up identity, networking, security, governance and cost controls **before** a single app deploys. It's the Cloud Adoption Framework's answer to 'how do we do this safely and repeatably, every time?'



### The headline you must internalise

Microsoft's own guidance: you do NOT need a separate 'AI landing zone'. AI is just another workload. You deploy Foundry / Azure OpenAI into an existing application landing zone — you just wire its networking, identity and data correctly.



### What happens without one

Ad-hoc deployments: keys in code, public endpoints, no egress control, no policy. AI amplifies both value and risk — without a foundation, the blast radius of one mistake is the whole tenant.

### Two kinds of landing zone



#### Platform landing zone

Shared services run by a central team: identity, connectivity, management subscriptions.



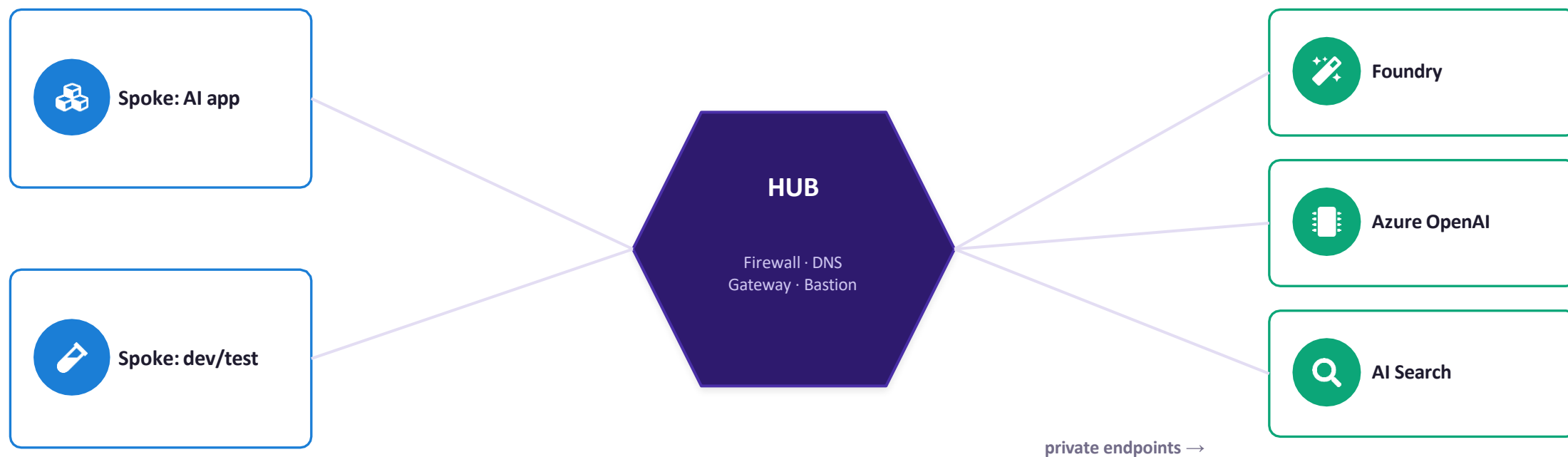
#### Application landing zone

One per workload, per environment. Your AI app lives here — dev, test and prod each get their own.

# STOP 4 • NETWORK EGRESS

## Hub-and-spoke: every packet leaves through one guarded door

Workloads live in spoke networks. All outbound traffic is forced through the hub's firewall — inspected, logged, policy-checked. AI services are reached over private endpoints, never the public internet.



### Egress is your job

Force all 0.0.0.0/0 traffic to the hub firewall — even subnets you don't expect to talk out. You own the data-exfiltration risk: the platform gives you the door, you must make everyone use it.

## The other two wires: who may act, and where data may live



### Identity-bound design



Entra ID is the perimeter. Every component gets a managed identity — App Service → Foundry, Foundry → Search/Storage.



Component-to-component calls use identity, not keys — the keyless pattern, enforced by the foundation.



Elevated roles via PIM: just-in-time, justified, time-boxed — no standing admin.



Conditional access + private DNS so identities resolve services privately.



### Data residency



Pin data with the Stop-1 dial: Regional or Data-Zone deployment scope + your chosen region.



Keep storage, Search and Cosmos in-region too — the model isn't the only place data sits.



Private endpoints keep that data off the public internet end-to-end.



Govern it with Azure Policy: deny non-compliant regions and public exposure by default.



TECHADEMY

STOP 5

# Managed Identities & RBAC



Never use API keys

*Identity is the perimeter. This is the whole game.*

## Never use API keys — the key is a shared password with no name on it

An API key is **a shared secret with broad permissions and no identity**. It tells the service WHAT to do but never WHO is asking. Leak it once — in a repo, a log, a notebook — and anyone holding it is you.



### You experienced this — now scale it up

In earlier session the lesson was: never put keys in the frontend; the API layer hosts the keys; the service-role key bypasses everything, so never expose it. The cloud version is simply the next step — don't hold keys at all. Managed identity is that principle, enforced by Azure.



### What identity gives you that a key can't

A name on every request (audit), least-privilege scope, automatic key rotation handled for you, and revocation by removing a role — not by scrambling to rotate a leaked secret across every service.

### Key vs identity



#### API key

Anonymous · broad · long-lived · leak = open door · manual rotation.



#### Managed identity

Named · least-privilege · short-lived tokens · auto-rotated · revoke via role.

## Managed identity + least-privilege RBAC + lock the back door



### 1 · Managed identity

Azure-hosted apps (App Service, AKS, VM, APIM) get a system- or user-assigned identity. No credential to handle.



### 2 · DefaultAzureCredential

One code path: az login locally, managed identity in prod. Same code, no keys, environment-aware.



### 3 · Least-privilege role

Assign 'Cognitive Services OpenAI User' for inference. 'Contributor' only for those who deploy/fine-tune.



### 4 · Disable local auth

Set disableLocalAuth = true and enforce with Azure Policy (Audit → Deny). Now keys simply don't work.

#### Grant least privilege — one CLI line

```
az role assignment create \
  --role "Cognitive Services OpenAI User" \
  --assignee-object-id $APP_IDENTITY \
  --scope /subscriptions/.../resourceGroups/rg-ai
```



### Watch the listKeys back door

A role that includes the listKeys permission lets a holder fetch the API key and sidestep all your RBAC. For human users, use a custom role modelled on the User role but with listKeys stripped out.



## Identity is the perimeter — four lines



### No keys

An API key is an anonymous shared password. Leak = full compromise. Don't hold them.



### Use managed identity

Named, least-privilege, short-lived, auto-rotated. Default Azure Credential everywhere.



### Least privilege

OpenAI User for inference; Contributor only to deploy. Strip listKeys from human roles.



### Lock the door

disableLocalAuth = true, enforced by Azure Policy. Make insecure deployments impossible.

*One sentence: the same instinct that kept keys out of the frontend now keeps them out of everything — identity replaces secrets.*



TECHADEMY

## STOP 6

# Azure OpenAI vs Direct OpenAI



Compliance · residency · cost · latency

*Where it all pays off — one decision, four lenses.*

## Same models, same labs — a very different envelope around them

| Lens           | Azure OpenAI (in Foundry)                                                      | OpenAI direct                                 |
|----------------|--------------------------------------------------------------------------------|-----------------------------------------------|
| Compliance     | HIPAA BAA, SOC 2, FedRAMP (Gov cloud), ISO — inside your Azure tenant          | SOC 2; narrower enterprise/compliance surface |
| Data residency | Global / Data-Zone / Regional + private endpoints; data stays in your boundary | Defaults to US; limited residency control     |
| Identity       | Entra ID, managed identity, RBAC, Private Link — keyless                       | API keys                                      |
| Cost           | ~Same per-token; PTU + EA/MACC drawdown; +15–40% enterprise add-ons            | Same per-token; simplest rate card            |
| Latency        | Predictable on PTU; latency SLA on Provisioned-Managed; in-region              | Best-effort; no SLA                           |
| Governance     | Azure Monitor, Purview, Policy, Cost Management built in                       | Vendor dashboard only                         |

*Same GPT models either way. You are choosing the ENVELOPE — compliance, residency, identity, governance — not the intelligence.*

## A simple rule you can give any team



### Choose Azure OpenAI when...

- ✓ You handle regulated or personal data (health, finance, government).
- ✓ Data residency or sovereignty is a hard requirement.
- ✓ You need keyless identity, Private Link and audit by default.
- ✓ You're already on Azure / an EA, and want one governance + billing model.
- ✓ You need a latency SLA or predictable cost at scale (PTU).



### Direct OpenAI can be fine when...

- ✓ It's a prototype, demo or internal tool with no sensitive data.
- ✓ You want the simplest possible setup and rate card.
- ✓ You need a brand-new model the day it launches, before Azure has it.
- ✓ Compliance, residency and governance are genuinely not in scope.
- ✓ You're not otherwise on Azure and don't need its envelope.



### The one-line rule

If compliance, data residency or identity-bound governance is in scope — which for any enterprise workload it almost always is — choose Azure OpenAI. Otherwise, direct OpenAI is a fine, simpler default for prototypes and non-regulated work.

## The hour in one checklist — and one thread



I know the three layers: Foundry platform · Azure OpenAI · supporting services



I decide PTU vs PAYG on 30–60 days of real telemetry



I read the SLA correctly: 99.9% availability ≠ latency or quality



AI lands in an existing application landing zone — private egress, private endpoints



I pick the deployment scope my compliance team needs — no stricter



Every component uses a managed identity; I never hold an API key



I right-size the model before I optimise anything else



Least-privilege RBAC + disableLocalAuth, enforced by Azure Policy

*The thread through all eight: identity. In the cloud, identity is the perimeter — design for it first.*



### The one idea to keep

Same models. The difference is the envelope —  
and the envelope is identity.



Foundry, Azure OpenAI, the landing zone and RBAC aren't four products — they're one identity-bound foundation. Build that, and AI runs in your enterprise without leaking data or losing control.

**Questions — and let's map it to a workload you own right now.**